Tutorial 2

1. Modify the following code to improve its cohesion.

```
public class Staff {
    public Speaker saveSpeaker(Speaker speaker);

    public boolean removeSpeaker(int speakerId);

    public Speaker findById(int speakerId);

    public void updateSpeakerPhoto(int speakerId, String photoUrl);

    public Product saveProduct(Product product);

    public void deleteProducts(List<Integer> productsId);

    public boolean deleteProduct(int productId);
}
```

2. Modify the following code to lower its coupling.

```
public class ParttimeStaff {
    public double getWorkedHours(){
        ....
    }

}

public class Payroll {
    private ParttimeStaff staff;

    public Payroll(ParttimeStaff ps){
        this.staff = ps;
    }

    public double CalculatePay(){
        return staff.getWorkedHours() * .......
    }
}
```

```
public class Car  {
    public void move() {
         System.out.println("Car is moving");
    }
}

class Traveler {
    Car c = new Car();
    public void startJourney() {
         c.move();
    }
}
```

3. According to SOLID principles, which is the most suitable situation to use inheritance?

4. A software system is considered to have a "good design" if the cost of changing it is minimized. Discuss how the concepts of Cohesion and Coupling contribute to achieving this goal.
5. A developer is designing a "Machine" hierarchy where different machines can have various combinations of movement (Fly, Swim) and combat capabilities (Laser, Rifle). Explain the "combinatorial explosion" problem that arises when using pure inheritance for this scenario and describe how using Composition can solve it.
6. According to SOLID principles, which is the most suitable situation to use composition?
7. Explain in detail the problem when the return type in an override method of a subclass is a long data type while the return type in the method of the superclass is an int data type?

8. Discuss the Single Responsibility Principle (SRP) and provide an example of how it can be applied in a software design.

9. How does the Open/Closed Principle (OCP) contribute to software maintainability and extensibility? Provide a real-world example to illustrate this principle.
10. Detail the Liskov Substitution Principle (LSP) and explain its significance in the context of inheritance and polymorphism.
11. Explain the Interface Segregation Principle (ISP) and discuss its role in software design.
12. Describe how the Dependency Inversion Principle (DIP) helps to decouple software modules and enhance flexibility in a software architecture.
13. What is the problem with the following interface? Improve

```
public interface FileStorage {
    public String storeFile(MultipartFile file, FileType fileType, Integer id);
    public Resource loadFileAsResource(String fileName, FileType fileType);
    public boolean removeFile(String fileName, FileType fileType);
    public DecodedJWT decodeJWT(String jwtString) throws Exception;
    public String GenerateJWT(UserDetails user);
    public String GenerateRefreshJWT(UserDetails user);

}
```

it.